

【专栏二：深度学习05】 - 【一张图讲清楚：梯度下降到底在干什么？】



快乐飞猪ing



· 码龄7年 · 公开 · 编辑



墨衍会员 · AI 创作全网分发

原创 · 2026-03-28 11:38:04 · 445 阅读



AI 写同款 · GEO 优化

完结撒花！吴恩达DeepLearning.ai《深度学习》课程笔记目录总集

吴恩达 Coursera DeepLearning.ai《深度学习》系列课程笔记目录总集

[阅读详情 >](#)

前言

梯度算出来以后，到底怎么让模型变好？

换句话说：模型是怎么一步步“少犯错”的？

这个问题的答案，就是：

- 梯度下降 (Gradient Descent)

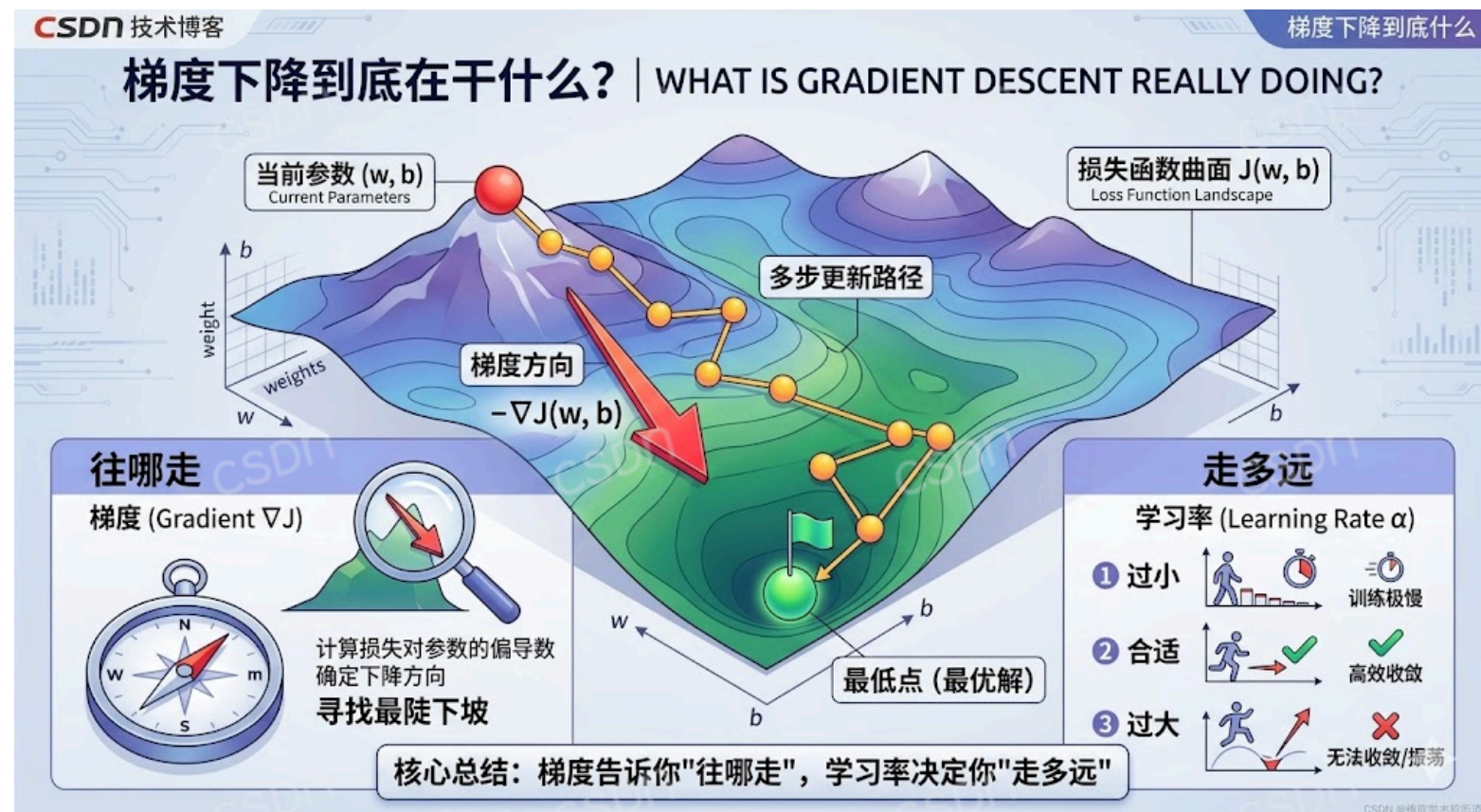
内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

把它放到一张图里看，其实很好理解，先看图。



结论：梯度下降其实只做两件事：

- 梯度告诉你“往哪走”
- 学习率决定你“走多远”

再通俗一点：

- 梯度负责告诉模型：朝哪个方向改参数，误差会下降
- 学习率负责告诉模型：这一步该多大

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

而整个训练过程，本质上就是：不断重复“找方向 → 走一步 → 再找方向 → 再走一步”，直到模型逐渐走到一个误差更低的位置。

一、先看图中间：为什么梯度下降经常被画成“下山”？

这张图中间最核心的部分，是一个起伏不平的曲面。

它表示的是：损失函数曲面 $J(w, b)$

这里的 $J(w, b)$ 可以先理解成：

- w ：权重
- b ：偏置
- J ：当前参数下，模型的总误差

也就是说，图上每一个位置，其实都代表一组参数 (w, b) ，而这组参数会对应一个误差值。

于是就有了一个很直观的比喻：

- 山顶附近 → 误差比较大
- 山谷附近 → 误差比较小

所以训练模型这件事被想象成：让参数从“误差高的地方”慢慢往“误差低的地方”走

这就是为什么大家总喜欢把梯度下降画成“下山”。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

二、图左上角：当前参数是什么意思？

图里左上角写着：当前参数 (w, b)

这表示：模型在训练的某一时刻，手里有一组**当前的参数值**。

你可以把它理解成模型当前的“**状态**”，比如：

- 一开始参数是随机初始化的
- 所以模型刚开始往往预测得很差
- 对应到图上，就是起点可能在山坡上，甚至山顶附近

这时候，模型要做的事情不是一下跳到终点，而是：**从当前参数出发，一步一步往更低误差的地方移动**

所以图上的那个红色大球，不是“最终答案”，而是：**训练此刻模型所在的位置**

三、图左下角：梯度到底在告诉我们什么？

图左下角这块，是整张图最关键的区域之一。

这里写的是：

- 往哪走 **梯度** (Gradient ∇J)

很多人第一次学梯度，最容易被符号吓到。

其实你先不用把它想得太数学化，可以先这样理解：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

梯度是在回答：如果我现在站在这里，往哪个方向走，**误差会上升得最快**？

注意，是“上升最快”。

也就是说：

梯度 ∇J 指向的是 误差上升最快的方向

那反过来， $-\nabla J$ 就是 误差下降最快的方向

这也是图里为什么会写：

- **梯度方向** $-\nabla J(w, b)$

更准确地说，这里其实是在表示：

- **下降方向 = 负梯度方向**

所以你可以把梯度理解成一个“**方向指示器**”：

- 它不是直接帮你到达终点
- 它只是告诉你：从当前位置出发，往哪边走更容易降误差

四、为什么说梯度像“指南针”？

图左下角专门画了一个指南针，这个比喻特别好。

因为梯度的作用，本质上真的很像指南针：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

- 它不负责帮你走路
- 它只负责告诉你方向

如果你现在站在山坡某个位置，想往低处走，最自然的方法就是：**看脚下坡往哪边最陡，就朝那边下去**
这就是梯度下降的直觉。

所以图里这句：**计算损失对参数的偏导数，确定下降方向，寻找最陡下坡**

你可以翻译成成人话理解成：

模型先看一眼：如果我现在稍微改一下参数，误差会怎么变？然后选一个“最容易把误差降下去”的方向。

五、先把公式讲明白：梯度下降到底怎么更新参数？

讲到这里，我们可以把那条最经典的更新公式放出来了： $\theta = \theta - \eta \nabla J(\theta)$

如果你暂时不想看 θ ，也可以拆成更具体的写法：

$$w := w - \alpha \frac{\partial J}{\partial w}$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

这里每一部分的意思其实都不复杂：

- w, b : 当前参数
- J : 损失函数
- $\frac{\partial J}{\partial w}, \frac{\partial J}{\partial b}$: 梯度, 也就是参数变化对误差的影响
- α : 学习率

这两个公式的核心意思就是：

新参数 = 旧参数 - 学习率 × 梯度

也就是说：

- 梯度告诉你往哪边改
- 学习率告诉你这一步改多大

这就和图里中间那条大红箭头完全对应上了。

六、图右下角：为什么“走多远”同样很重要？

光知道方向还不够，你还得决定：这一脚迈多大

这就是图右下角讲的内容：

- 走多远，学习率 (Learning Rate α)

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页: https://blog.csdn.net/m0_46279060

学习率的作用特别好理解：它决定每一次参数更新的步长。

图里把学习率分成了三种情况，这个非常标准：

1、学习率过小：训练很慢

如果学习率太小，就会出现什么情况？

- 每次虽然方向对了
- 但只敢挪一点点
- 结果就是走得特别慢

对应到图上，就是你明明知道山谷在哪边，但每次只挪半步。

那当然会下降得很慢。

所以图里写：

- 过小 → 训练极慢

2、学习率合适：高效收敛

如果学习率比较合适：

- 每一步方向是对的
- 步子也不大不小
- 模型就能比较稳定地往低误差区域靠近

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

这就是最理想状态，对应图里的表述：

- **合适 → 高效收敛**

也就是：**走得稳，而且走得快**

3. 学习率过大：可能震荡甚至发散

这个也是初学者特别容易忽略的一点，很多人会想：既然学习率小会慢，那我干脆把学习率调大一点，不就更快到了吗？

问题是：**步子太大，可能会直接跨过最低点。**

甚至会出现：

- 左边走过头
- 再往右边改过头
- 来回震荡
- 最后根本停不下来

所以图里写：

- **过大 → 无法收敛 / 振荡**

这也正是学习率最难调的地方之一。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

七、图中间的“多步更新路径”到底在表达什么？

图中央那一串小黄球和折线路径，其实特别关键。

它表达的是：

- 梯度下降不是一步到位，而是很**多次迭代的累积**。

也就是说，模型不是一下子突然变聪明的。

它的进步方式更像这样：

- **先找一个方向**
- **走一步**
- **到了新位置，再重新算梯度**
- **再走一步**
- **重复很多次**

所以图里才会写：**多步更新路径**

这条路径本质上就是训练过程本身，你可以把它理解成：

- **模型每一轮训练，都在“少犯一点错”**

这也是为什么我一直觉得，梯度下降最适合的直觉理解不是“求最优”，而是：

- **不断往更不犯错的方向移动**

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

八、图下方的绿色旗子：为什么不是一步走到“最低点”？

图里最底部放了一个绿色旗子，旁边写着：

- **最低点（最优解）**

这在科普图里是可以的，因为它能帮助读者理解训练目标。

但如果我们稍微严谨一点来讲，更准确的理解应该是：

- **梯度下降的目标，是尽量找到误差更低的位置**

在真实深度学习训练里，事情往往没有“完美最低点”那么简单：

- **损失曲面可能很复杂**
- **可能有很多局部低点**
- **也可能有平坦区域、鞍点**

所以你可以把图里的绿色旗子理解成：**一个更优的位置，一个误差更低的区域**

而**不一定**非要把它想成“**全局唯一最优**”。

不过这不影响这张图的核心表达，作为科普图，它已经足够清楚了。

九、把这张图翻译成伪代码，其实就是这样

如果你喜欢从代码角度理解，这张图对应的流程可以压缩成下面这几行：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

```
1 | params = init()
2 |
3 | for step in range(N):
4 |     y_pred = model(x, params)
5 |     loss = compute_loss(y_pred, y_true)
6 |
7 |     grad = compute_gradient(loss, params)
8 |
9 |     params = params - lr * grad
```

你会发现，它和图完全一致：

- `loss` → 对应损失函数曲面
- `grad` → 对应梯度方向
- `lr` → 对应学习率
- `params = params - lr * grad` → 对应每一步更新路径

所以你也可以这样理解：

梯度下降不是一个很玄的概念，它本质上就是“算梯度，然后按这个方向改参数”。

十、为什么梯度下降能让模型越来越好？

现在我们把整张图串起来，你就会发现它讲的是一个非常朴素的逻辑：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

- 当前参数先做一次预测
- 预测和真实值有误差
- 误差形成损失
- 梯度告诉模型：往哪边改更能减少损失
- 学习率决定改多少
- 改完再来一轮

于是模型就会：

- 一开始误差很大
- 后来越来越小
- 参数越来越合理
- 预测越来越接近真实值

所以说到底：

模型不是因为“理解了数据”才变强，而是因为它不断在往“更少犯错”的方向调整。

这就是梯度下降最本质的意义。

十一、一句话总结整张图

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060

如果让我只用一句话来解释这张图，我会这样说：

梯度下降的本质，就是利用梯度这个“方向指示器”，再配合学习率这个“步长控制器”，让模型参数在损失曲面上一点点走向更低误差的位置。

很多人第一次学梯度下降时，会觉得它是一个很抽象的[优化算法](#)。

但如果把它放进这张图里，其实你会发现它特别朴素：

- 先看哪里错了
- 再决定往哪边改
- 然后迈一小步
- 重复很多次

思考

如果梯度方向是对的，但学习率设置得特别大，模型会更快到达最优点，还是更容易来回震荡？为什么？

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159548192

作者主页：https://blog.csdn.net/m0_46279060